

DEDICATED TO INNOVATION

www.bacancy.com





**Customer Satisfaction
Is Our Highest Priority**



Employee Matters



In this course

What we are learning in this session

- ▶ Introduction
- ▶ Data Types
- ▶ Naming Conventions
- ▶ Tables Relations
- ▶ Keys

Let's Start

SQL Server - Introduction

- ▶ SQL Server is a relational database management system, or RDBMS, developed and marketed by Microsoft.
- ▶ Similar to other RDBMS software, SQL Server is built on top of SQL, a standard programming language for interacting with relational databases.
- ▶ SQL Server is built for the basic function of storing retrieving data as required by other applications. It can be run either on the same computer or on another across a network.

SQL Server - Data Type

- ▶ In SQL Server, data type specifies the type of data that can be stored in a column of a table such as integer data, string data, date & time, binary strings, etc.
- ▶ Assigning an appropriate data type to columns in a table is crucial while designing a database. It affects the performance and efficiency of the database and the application using the database.

Category	Data Types
Exact numerics	bit, tinyint, smallint, int, bigint, decimal, numeric, money, smallmoney
Approximate numerics	Real, Float
Date & Time	date, smalldatetime, datetime, datetime2, datetimeoffset, time
Character strings	char, varchar, text
Unicode Character strings	nchar, nvarchar, ntext
Other data types	cursor, hierarchyid, sql_variant, spatial Geometry types, spatial Geography types, rowversion, uniqueidentifier, xml, table

- ▶ SQL Server provides built-in data types for all kinds of data that can be used within SQL Server.

SQL Server - Naming Conventions

- ▶ SQL Server defines a set of rules (dos and don'ts) for naming SQL Server Objects called naming convention, but also gives the user to follow their own preferred style. It is advisable to follow a single naming convention for all database objects consistently.
- ▶ **Why Use Naming Conventions?**
 - ❖ Naming convention for tables, columns, and all other related database objects like views, stored procedures, indexes, triggers, etc., are important for the success and maintenance of a project. A database can have many tables and users working on it.
 - ❖ By following a naming convention, you can spend less time finding what you need and helps in efficient database management.
- ▶ **SQL Server Database Object Naming Rules:**
 - ❖ Names of database objects such as tables, columns, stored procedures, views, etc, can contain alphanumeric characters and must begin with an alphabet or an underscore.
 - ❖ Case sensitivity is determined by the settings of the database in which the object is created. By default, SQL Server is not case sensitive.
 - ❖ A name that starts with the 'at' (@) sign always denotes a local variable or parameter and cannot be used as a name for any other type of object.

SQL Server - Naming Conventions

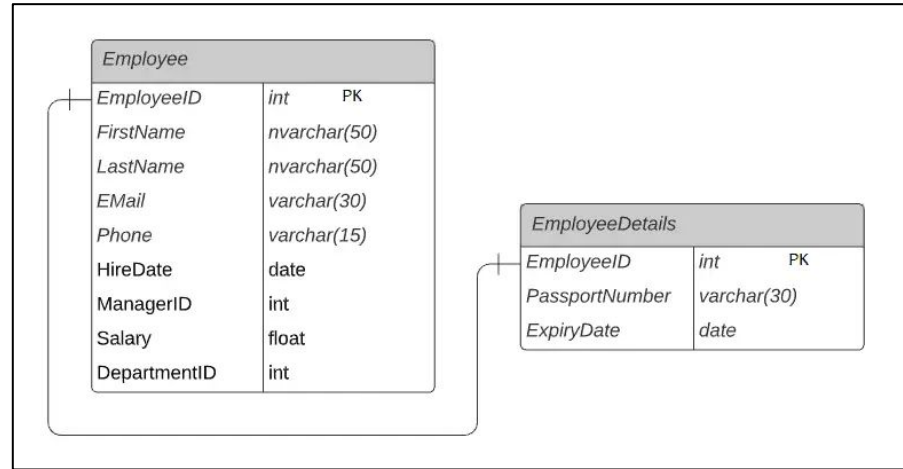
- ❖ A name that begins with the number sign (#) denotes a temporary table or procedure.
- ❖ The Object name should not be a SQL reserved word. SQL Server reserves both lowercase and uppercase versions of reserved words.
- ❖ Some system procedures and functions are given prefixes like 'sp_', 'xp_', or 'dt_' to signify that they are system objects and should be searched for in the master database. So it is better to avoid using these prefixes while creating database objects.
- ❖ There are a few commonly accepted styles to name a table & column in a database. It can be all 'lower case', 'UPPERCASE', 'Pascal Case' or 'camel Case'.
- ❖ Avoid using spaces between words for table names and columns as they will require square brackets and confuse scripts and procedures. Instead, use an underscore for joining multiple words. For example, EMPLOYEE_SKILL, employee_id instead of [EMPLOYEE ID].
- ❖ Avoid using prefixes like 'tbl_' for table names.
- ❖ While naming columns, use nouns that describe what is stored in the column.
 - Example: FIRST_NAME, ADDRESS, EMAIL, PHONE. Avoid using words that are just data types like text or date.

SQL Server - Tables Relations

- ▶ It is important to understand and design relationships among tables in a relational database like SQL Server. In a relational database, each table is connected to another table using the Primary-Foreign Key constraints.
- ▶ **Table relationships in SQL Server database are of three types: One-to-One, One-to-Many, and Many-to-Many**

- ▶ **1). One-to-One:** In One-to-One relationship, one record of the first table will be linked to zero or one record of another table.

For example, each employee in the **Employee** table will have a corresponding row in **EmployeeDetails** table that stores the current passport details for that particular employee.

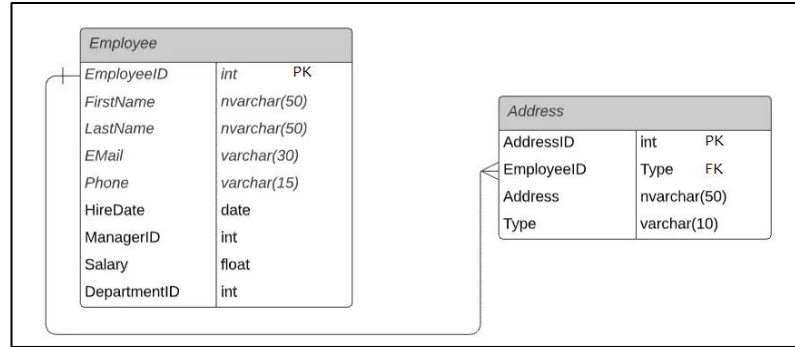


- ▶ Each employee will have zero or one record in the **EmployeeDetails** table. This is called zero or one-to-one relationship.

SQL Server - Tables Relations

- ▶ Above, **EmployeeID** column is the primary key as well as foreign key column in the **EmployeeDetails** table that linked to **EmployeeID** of the **Employee** table. This forms zero or one-to-one relation.
- ▶ **2). One-to-Many Relation:** One-to-Many is the most commonly used relationship among tables. A single record from one table can be linked to zero or more rows in another table.
- ▶ Let's take an example of the **Employee** and **Address** table in the database. The **Employee** table stores employee records where **EmployeeID** is the primary key. The Address table holds the addresses of employees where **AddressID** is a primary key and **EmployeeID** is a foreign key. Each employee will have one record in the **Employee** table. Each employee can have many addresses such as Home address, Office Address, Permanent address, etc.
- ▶ The **Employee** and **Address** tables are linked by the key column **EmployeeID**. It is a foreign key in the Address table linking to the primary key **EmployeeID** in the **Employee** table. Thus, one record of the **Employee** table can point to multiple records in the Address table. This is a One-to-Many relationship.

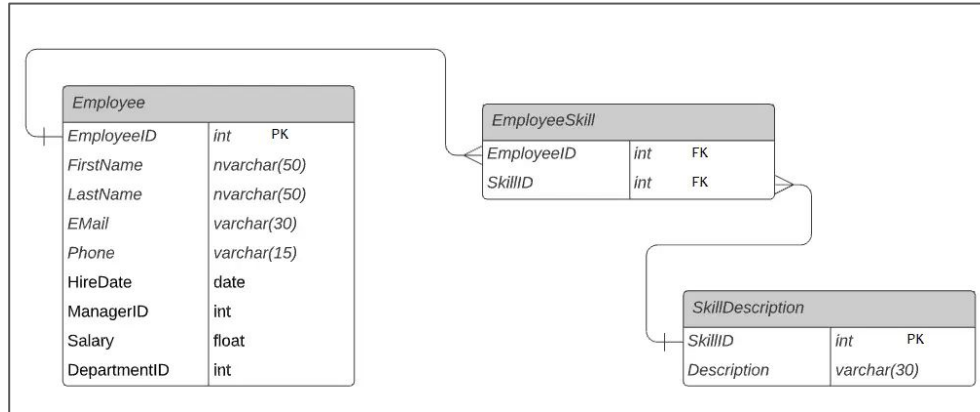
SQL Server - Tables Relations



- ▶ **3). Many-to-Many Relation:** Many-to-Many relationship lets you relate each row in one table to many rows in another table and vice versa. As an example, an employee in the **Employee** table can have many skills from the **EmployeeSkill** table and also, one skill can be associated with one or more employees.
- ▶ The following figure demonstrates many-to-many relation between **Employee** and **SkillDescription** table using the junction table **EmployeeSkill**.

SQL Server - Tables Relations

- ▶ Every employee in the **Employee** table can have one or many skills. Similarly, a skill in the **SkillDescription** table can be linked to many employees. This makes a many-to-many relationship.



- ▶ In the example above, the **EmployeeSkill** is the junction table that contains **EmployeeID** and **SkillID** foreign key columns to form many-to-many relation between the **Employee** and **SkillDescription** table.

SQL Server - Keys

- ▶ **1). Primary Keys:** In SQL Server, a Primary key is a constraint that uniquely identify each row in the table. It enforce data integrity in tables.
 - A table can have only one primary key.
 - A primary key can be defined on one column or the combination of multiple columns known as a composite primary key.
 - The primary key uniquely identifies each row in a table. It is often defined on the identity column.
 - The Primary key column do not allow NULL or duplicate values. It will raise an error if try to do so.
 - All columns defined within the primary key constraint must be defined as a NOT NULL column.
 - If clustered or nonclustered is not specified, then a unique clustered index for the primary key column will be created if there no clustered index on the table. This makes retrieving data faster whenever the primary key column is included in the query.
- ▶ **Create a Primary Key:** A primary key can be configured in the create table SQL script. Append **CONSTRAINT** statement at the end of all column declaration.

SQL Server - Keys

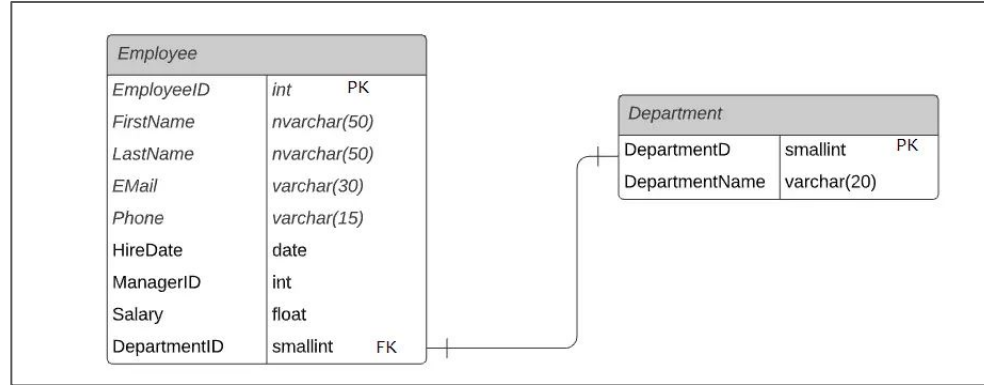
▶ **Syntax:** `CONSTRAINT <primarykey_name> PRIMARY KEY [CLUSTERED | NONCLUSTERED] (<column_names>)`

- ▶ The name of a primary key should be in PK_TableName_ColumnName format to recognize it easily.
- ▶ **Create a Primary Key in an Existing Table:** Use the **ALTER TABLE** statement to add a primary key in an existing table.

▶ **Syntax:** `ALTER TABLE Employee
ADD CONSTRAINT PK_Employee_EmployeeID PRIMARY KEY (EmployeeID)`

- ▶ **2). Foreign Keys:** The foreign key establishes the relationship between the two tables and enforces referential integrity in the SQL Server. For example, the following Employee table has a foreign key column DepartmentID that links to a primary key column of the Department table.
 - A foreign key column can be linked to a primary key or a unique key column of the same or another table.
 - The table having the foreign key constraint is called the child table, and the table being referenced by the foreign key is called the parent table. E.g. Employee is a child table and Department is a parent table.

SQL Server - Keys



- A value other than NULL is entered in the column of the foreign key constraint, that value must already exist in the referenced column of the parent table. Else you will get a foreign key violation error.
- A foreign key constraint defined at the table level (on a combination of columns) should have the same number of reference columns as the number of columns defined in the constraint list. The data type of each column in the constraint must be the same as the corresponding column in the column list.
- There is no limit on the number of foreign key constraints a table can contain that references other tables. However, it is limited by the hardware configuration and the database design.

SQL Server - Keys

- Foreign key constraints are not enforced on temporary tables.
- ▶ **3). Unique Keys:** The Unique Constraint ensures the uniqueness of all values and no duplicate values are entered in a column of a table.
- ▶ The unique constraints can be created using CREATE or ALTER TABLE SQL. Add unique constraints after columns definitions in the CREATE TABLE statement.
- ▶ **Syntax:** **CONSTRAINT** <constraint_name> **UNIQUE**(<column_name>)
- ▶ **Important Points:**
 - Both Unique constraint and Primary key constraint enforce uniqueness. It is recommended to use Unique constraint instead of Primary key constraint whenever you want to enforce uniqueness in a column.
 - Unlike Primary key constraint, Unique constraints allow only one NULL value.
 - A unique index is automatically created when a unique key constraint is created.
 - SQL Server raises an error whenever a duplicate value is inserted or updated.

SQL Server - Keys

▶ Important Points:

- When a Unique index is added to an existing column in a table, the database engine first checks for the uniqueness of the specified column data. If a duplicate entry is found, then the engine returns an error and does not add the constraint.
- A Unique key in a table can be referenced by a Foreign Key from another table.

Thank you